GameCraft

A Minecraft-Themed Multi-Game Hub

CS108 — Systems & Software Lab

Project Report

Authors: Akshita Jain (25B0999)

Maulik Jain (25B1002)

Date: April 29, 2026

Department of Computer Science and Engineering
IIT Bombay

Contents

1 External Libraries Used **2**

2 Features Implemented **2**
2.1 Authentication System (Bash) 2
2.2 Main Hub (Python / Pygame) 2
2.3 Game: Tic-Tac-Toe 3
2.4 Game: Othello (Reversi) 3
2.5 Game: Connect Four 4
2.6 League Mode 4
2.7 Leaderboard & Analytics 4

3 Code Logic **6**
3.1 Generic `Game` Base Class (`game.py`) 6
3.2 Tic-Tac-Toe (`tictactoe.py`) 6
3.3 Othello (`othello.py`) 7
3.4 Connect Four (`connect4.py`) 7
3.5 Authentication System (`main.sh`) 7
3.6 Leaderboard (`leaderboard.sh`) 7

4 Hurdles Faced **9**

5 What We Would Do Differently **10**

6 Bibliography **11**

1 External Libraries Used

Library	Purpose
`pygame`	Core graphics and audio library. Handles window creation, event polling, sprite and font rendering, image loading, and audio playback via `pygame.mixer`.
`numpy`	Represents game boards as 2D arrays and evaluates win conditions via vectorised array slicing without manual loops.
`matplotlib`	Generates post-match charts: a pie chart of game popularity and a bar chart of top players by wins, saved as PNGs and loaded back into Pygame.
`sys`	Provides `sys.argv` to receive authenticated player names from the Bash entry point, and `sys.exit()` for clean termination.
`os`	Constructs cross-platform paths, invokes Bash subprocesses via `os.system()`, and checks file existence with `os.path.exists()`.
`math`	Supplies `math.sqrt()` and trigonometric functions used in rendering.
`csv`	Parses `history.csv` for cumulative win/loss records appended after every game session.

2 Features Implemented

2.1 Authentication System (Bash)

The application launches exclusively through `main.sh`, enforcing two-player authentication before any game session begins.

- **SHA-256 Password Hashing:** Passwords are never stored in plaintext; they are hashed with `sha256sum` and compared against `users.tsv`.
- **Registration:** If a username is not found, the user is prompted to register; a new tab-separated entry is appended to `users.tsv`.
- **Uniqueness Constraint:** Both players must authenticate as different users; Player 2 is re-prompted on a duplicate login.
- **Argument Passing:** On successful dual-login, `python3 game.py $PLAYER1 $PLAYER2` is invoked, passing authenticated usernames as command-line arguments.

2.2 Main Hub (Python / Pygame)

The Pygame engine presents a Minecraft-themed graphical hub with multiple screens.

- **Start Screen:** Displays the "GameCraft" title and both player names. Each player selects a Minecraft character avatar (Zombie, Pig, Dog, or Steve) from side panels before proceeding.
- **Game Menu:** Lists three playable games and a League mode; navigation back to the start screen is available.
- **How to Play:** A dedicated screen with instructions overlaid on a thematic background image.
- **Statistics Screen:** Displays Matplotlib-generated charts (game popularity pie chart, top-players bar chart) loaded as Pygame surfaces.
- **Leaderboard Screen:** Players select a game and sort method (Total Wins, Total Losses, W/L Ratio) and trigger a formatted terminal display via `leaderboard.sh`.

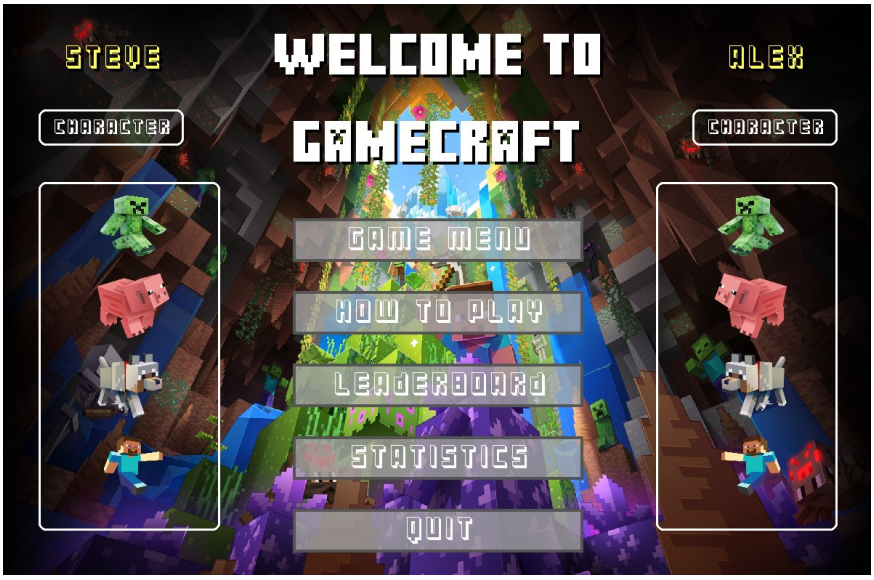


Figure 1: Start Screen — "Welcome to GameCraft" with avatar selection panels

2.3 Game: Tic-Tac-Toe

Played on a 10×10 board; 5 consecutive marks are required to win.

- Players place Diamond (Player 1) and Golden Apple (Player 2) sprites alternately.
- Win conditions checked horizontally, vertically, and diagonally using NumPy slicing.
- A win triggers an animated line drawn through the winning sequence.
- Ghost piece previews shown on hover before placement.

2.4 Game: Othello (Reversi)

Standard 8×8 board with classic Othello rules.

- The board initialises with two Black and two White discs at the centre.
- Valid move detection searches all 8 directions for opponent pieces that would be flanked.

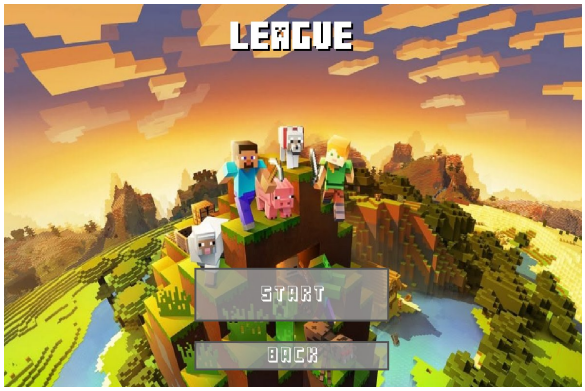
- All flanked opponent pieces are flipped on a valid move.
- A player's turn is skipped if they have no valid moves; the game ends when neither player can move.

2.5 Game: Connect Four

7×7 vertical grid with gravity-based piece dropping.

- Players drop Diamond vs. Emerald sprites; pieces fall to the lowest available row.
- Win detection checks for 4 in a row horizontally, vertically, and diagonally via NumPy.
- Ghost piece preview and a mid-game pause screen are available.

(a) Tic-Tac-Toe (10×10 board)



(b) League mode start screen

Figure 2: In-game screens

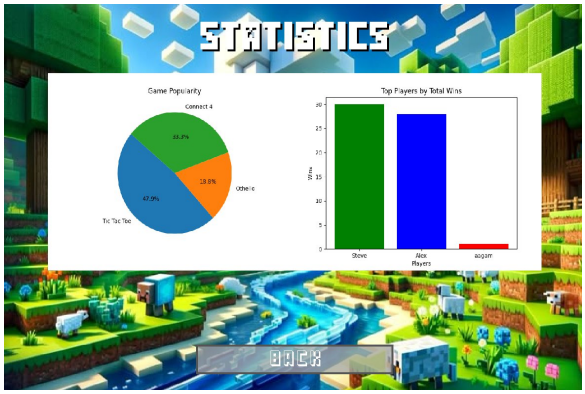
2.6 League Mode

Runs all three games sequentially in a single session.

- Results of Tic-Tac-Toe, Othello, and Connect 4 are tallied.
- The player with more individual game wins is declared the League Champion.
- A final champion screen displays the score (e.g. 2–1) and the winner's name.

2.7 Leaderboard & Analytics

- `update_history()` in `game.py` appends results to `history.csv` after each match.
- `leaderboard.sh` operates in two modes: `update` (writes new records) and `display` (sorts and renders a terminal table via `column -t`).
- Matplotlib refreshes a game-popularity pie chart and a top-players bar chart after every match.



(a) Statistics screen (Matplotlib charts)

(b) Leaderboard selection screen

Figure 3: Analytics and leaderboard screens

3 Code Logic

3.1 Generic `Game` Base Class (`game.py`)

The `Game` class is the abstract foundation inherited by all three game modules.

Listing 1: Game base class structure

```
class Game:
    def players(self):
        self.p1 = sys.argv[1] if len(sys.argv) > 1 else "Steve"
        self.p2 = sys.argv[2] if len(sys.argv) > 2 else "Alex"

    def switch_turns(self):
        self.player *= -1   # Player 1 = 1, Player 2 = -1

    def reset(self):
        self.board = np.zeros(self.board_shape)
        self.player = 1
        self.game_over = False

    def is_empty(self, row, col):
        return self.board[row, col] == 0

    def is_full(self):
        return np.all(self.board != 0)

    def check_win(self):
        pass  # Overridden by each subclass
```

The board is always a NumPy array of zeros: `+1` encodes Player 1's pieces and `-1` encodes Player 2's. `switch_turns()` multiplies the current player value by -1. `check_win()` is a stub overridden by each subclass.

3.2 Tic-Tac-Toe (`tictactoe.py`)

`TicTacToe` extends `Game` with a 10×10 board and a win threshold of 5. Rather than iterating over every 5-cell window, the board is sliced and four shifted copies are summed. Any cell equal to $5 \times$ `player` indicates a win in that direction.

Listing 2: Vectorised horizontal win check

```
horizontal = (self.board[:, :-4] + self.board[:, 1:-3]
              + self.board[:, 2:-2] + self.board[:, 3:-1]
              + self.board[:, 4:])
rows, cols = np.where(horizontal == 5 * self.player)
```

The same technique applies for vertical, main-diagonal, and off-diagonal directions. Winning cell coordinates are stored in `self.winning_line` for the animated win-line renderer.

3.3 Othello (`othello.py`)

Move Validation. `switch_possible(row, col, dr, dc, player)` walks in direction `(dr, dc)`, counting consecutive opponent pieces, and returns `True` only if a friendly piece closes the bracket.

Piece Flipping. `switch_pieces(row, col, player)` iterates over all 8 directions, setting `board[r][c] = player` for every traversed opponent cell where flipping is valid.

End Condition. The game ends when the board is full or neither player can move. Piece counts are compared with `np.sum(board == player)`.

3.4 Connect Four (`connect4.py`)

Listing 3: Gravity: finding the lowest empty row

```
def get_available_row(self, col):
    indices = np.where(self.board[:, col] == 0)[0]
    if len(indices) > 0:
        return indices[-1]   # bottom-most empty row
    return None
```

Win detection uses the same shifted-slice approach with a sum threshold of 4, checking `== 4` for Player 1 and `== -4` for Player 2.

3.5 Authentication System (`main.sh`)

Listing 4: SHA-256 password hashing and verification

```
hash_password() {
    echo -n "$1" | sha256sum | awk '{print $1}'
}

STORED_HASH=$(grep -P "^${USER_NAME}\t" "${FILE}" | awk '{print
   $2}')
ENTERED_HASH=$(hash_password ${PASSWORD})

if [[ "${STORED_HASH}" == "${ENTERED_HASH}" ]]; then
    echo "Login successful"
fi
```

`echo -n` suppresses the trailing newline for consistent hashing. `grep -P` with a tab-anchored pattern ensures exact username matching. A uniqueness loop prevents both players from using the same account.

3.6 Leaderboard (`leaderboard.sh`)

The script operates in two modes via `$1`. In **update** mode, existing records are loaded into Bash associative arrays keyed by `"$user,$game"`, win/loss counts are incremented, and `history.csv` is rewritten from scratch. In **display** mode, records for the requested game are filtered, a W/L ratio is computed with `bc` for floating-point arithmetic, and the

result is piped through `sort` and `column -t -s,` for a clean terminal table. Undefeated players receive a large integer ratio to represent infinity.

4 Hurdles Faced

NumPy Win Condition Design. The specification required win conditions via NumPy array slicing exclusively. We overcame the initial unfamiliarity by realising that summing shifted board slices counts consecutive same-sign values in one vectorised operation; once understood for horizontal wins it extended naturally to all other directions.

Pygame Coordinate System vs. Grid Indices. Pygame uses pixel coordinates while the game board uses row/column indices. Converting mouse click positions with non-zero offsets required careful arithmetic. We centralised all offsets in `configuration.py` and used them consistently across modules.

Inter-Module State Management. Launching sub-games from the hub's main loop and returning a result string required managing which Pygame screen and clock belonged to each module. We resolved this by passing the shared `screen` surface as a parameter to each game's `main()` function.

Bash SHA-256 Hashing with Special Characters. Passwords containing spaces or special shell characters caused inconsistent hashes. Using `echo -n` and careful variable quoting fixed this; `read -s` was added for silent password input.

Bash Associative Arrays and CSV Parsing. Updating cumulative records without a database required associative arrays keyed by `"$user,$game"`. The challenge was re-serialising them to CSV without duplicates. We solved this by loading the full CSV into memory and rewriting it entirely after each update.

Matplotlib Charts inside Pygame. `plt.savefig()` would silently overwrite incorrectly if the previous figure was not closed. We added `plt.close()` after every `savefig()` and wrapped all plot generation in `refresh_plots()`, called after every match.

Othello Valid Move Skipping. Checking all 64 cells for a valid placement each turn is an $O(n^2 \cdot d)$ operation. We implemented `has_any_valid_move(player)`, which short-circuits on the first valid cell found, keeping performance acceptable.

5 What We Would Do Differently

Online Multiplayer via Sockets. The current architecture requires both players on the same machine. We would explore Python's `socket` library to allow two players on different machines to connect over a local network, serialising the NumPy board array with `json` or `pickle` each turn.

Animated Piece Flipping in Othello. Piece flips currently happen instantaneously. A brief flip animation would make it easier to track which pieces were just flipped and make the game feel more polished.

Sound Effects for Each Game. Background music plays for the hub, but individual games lack contextual audio feedback. We would add click sounds for piece placement, a sound for illegal moves, and celebratory audio for wins using `pygame.mixer.Sound`.

Password Reset & Account Management. The Bash authentication has no mechanism for a forgotten password. We would implement a security-question-based reset flow in `main.sh`, storing the question and its hashed answer as additional fields in `users.tsv`.

Refactor into MVC Architecture. Game logic and rendering are currently mixed, particularly in `renderer.py`. We would cleanly separate Model (NumPy board state), View (Pygame rendering), and Controller (event handling) in each game module to improve extensibility and testability.

6 Bibliography

1. Pygame-CE Official Documentation. https://pyga.me/docs/
2. Pygame Sprite Module Reference. https://www.pygame.org/docs/ref/sprite.html
3. NumPy Documentation — Array Indexing and Slicing. https://numpy.org/doc/stable/reference/arrays.indexing.html
4. Matplotlib Documentation — `pyplot` API. https://matplotlib.org/stable/api/pyplot_summary.html
5. Python `os` Module Documentation. https://docs.python.org/3/library/os.html
6. Bash Reference Manual — Associative Arrays. https://www.gnu.org/software/bash/manual/bash.html#Arrays
7. SHA-256 hashing in Bash using `sha256sum`. https://linux.die.net/man/1/sha256sum
8. `bc` — Arbitrary precision calculator language. https://linux.die.net/man/1/bc
9. Othello (Reversi) Official Rules. https://www.worldothello.org/about/about-othello/othello-rules/official-rules/english